

NXT CRM DB 스키마

NXT CRM v2 — DB 스키마

정본: `supabase/schema.sql` (운영 DB `ghuevnxgcdltgupoddsn` 추출본). 정확한 DDL은 `schema.sql` 기준. DB 엔진: PostgreSQL (Supabase). 확장 `pg_trgm`. 타입은 PostgreSQL 기준 표기.

설계 원칙

- 통합 계약 모델: `contracts` 한 테이블에 `msp/edu/dev`를 `type`으로 구분, MSP만 `contract_msp_details`로 1:1 확장.
- 고객도 동일: `clients` + `client_msp_details` / `client_edu_details` 타입별 1:1 확장.
- Soft delete: 삭제 대신 `deleted_at` 기록. 조회는 `deleted_at IS NULL`.
- RLS 전면 적용: `can_access_client()` / `can_access_contract()`가 팀↔사업유형 도메인 매핑으로 행 접근을 판정.
- 예치금 잔액 = 거래 합산 트리거: `deposit_transactions` 변경 시 `deposit_accounts.balance` 자동 재계산.
- `profiles` (로그인 계정) ≠ `employees` (직원 마스터): 영업·기술 담당은 `employees`를 참조.
- 고객 계층 2단계 제한 (`parent_id`, 트리거 강제).
- 표시 ID 자동 발번: `client_id` (UNIV-001…), `contract_id` (MSP-001 / CT2026001).

ERD 개요

[팀·권한·사용자]

`teams` —< `profiles` —< `user_preferences`

└< `employees`

`team_business_domains` (팀유형 ↔ 사업유형 매핑 · FK 아님 · 접근제어 전용)

[고객]

`clients` —1 `client_msp_details`

└1 `client_edu_details`

└< `contacts`

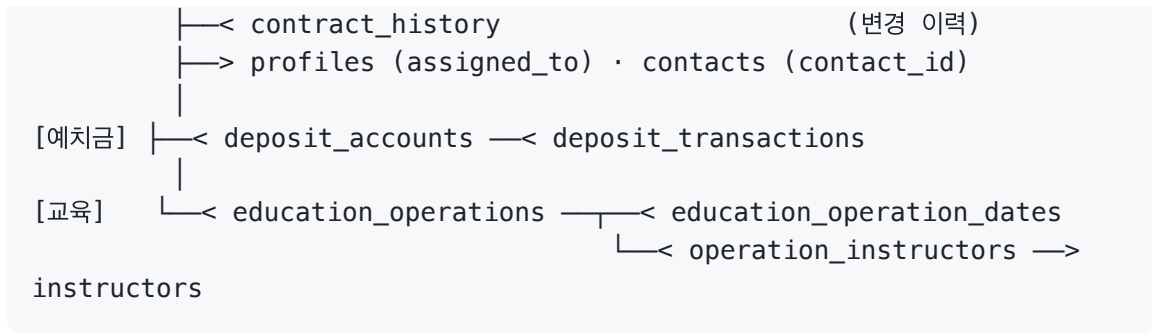
└< `clients` (parent_id · 2단계)

└< `contracts` (고객의 계약)

[계약] └1 `contract_msp_details` —> `employees` (`sales_rep_id`)

└< `contract_teams` —> `teams` (매출 배분 %)

└< `contract_tech_leads` —> `employees` (담당 기술)



기호: —< 1:N · —1 1:1 · —> 참조(FK)

1. teams — 조직 팀

컬럼	타입	제약	설명
id	uuid	PK	
name	text	NOT NULL, UNIQUE	팀 표시명
type	team_type	NOT NULL	mvp / tt / dev / ops / ai / ptn (tt=Technical Training)
created_at	timestampz	NOT NULL, default now()	

2. team_business_domains — 팀유형 ↔ 사업유형 매핑

컬럼	타입	제약	설명
team_type	team_type	PK	
business_type	business_type	PK	

시드: (ops,edu)(ops,mvp)(tt,edu)(dev,dev)(ai,mvp)(ptn,mvp)

접근 제어 핵심. can_access_* 함수가 “내 팀유형이 접근 가능한 사업유형”을 이 표로 판정. FK가 아니라 enum 값 조합 매핑.

3. profiles — 인증 사용자 (auth.users 1:1)

컬럼	타입	제약	설명
id	uuid	PK, FK → auth.users (CASCADE)	Supabase 인증 유저 id
name	text	NOT NULL	

컬럼	타입	제약	설명
email	text	NOT NULL, UNIQUE	
role	user_role	NOT NULL, default 'staff'	staff / team_lead / admin / c_level
team_id	uuid	FK → teams	소속 팀
position	text	NULL	직책
created_at / updated_at	timestampz	NOT NULL	

신규 `auth.users` 생성 시 `handle_new_user()` 트리거가 profiles 행을 자동 생성.

4. employees — 직원 마스터 (영업·기술 담당 지정용)

컬럼	타입	제약	설명
id	uuid	PK	
name	text	NOT NULL	
email / phone / position	text	NULL	
team_id	uuid	FK → teams	
profile_id	uuid	FK → profiles	로그인 계정 연결(있으면)
is_active	boolean	default true	
is_sales_rep	boolean	NOT NULL, default false	영업 담당 후보 여부
created_at / updated_at	timestampz	NOT NULL	

인덱스: (team_id)

profiles (로그인)와 별개. 계약의 영업/기술 담당은 employees를 참조한다.

5. user_preferences — 사용자별 UI 환경설정

컬럼	타입	제약	설명
id	uuid	PK	
user_id	uuid	NOT NULL, UNIQUE, FK → profiles (CASCADE)	
preferences	jsonb	NOT NULL, default '{}'	컬럼 표시 설정 등

컬럼	타입	제약	설명
updated_at	timestampz	NOT NULL	

RLS: 본인(`user_id = auth.uid()`)만 접근.

6. `clients` — 고객 (CRM 자산, soft delete)

컬럼	타입	제약	설명
id	uuid	PK	
client_id	text	NOT NULL, UNIQUE	표시 ID (<code>UNIV-001</code> 등, <code>generate_client_id</code>)
name	text	NOT NULL	
client_type	client_type	NOT NULL	univ / corp / govt / asso / etc
grade	client_grade	NULL	A~E
business_types	business_type[]	default '{}'	소속 사업유형(접근 제어에 사용)
parent_id	uuid	FK → clients (SET NULL)	상위 고객 (계층 2단계, 트리거 강제)
status	client_status_type	NOT NULL, default '상태없음'	
memo	text	NULL	
created_at / updated_at	timestampz	NOT NULL	
deleted_at	timestampz	NULL	soft delete

인덱스: (id) WHERE deleted_at IS NULL · (name) gin_trgm (부분 검색) · (parent_id) WHERE parent_id IS NOT NULL

고객은 삭제하지 않는다. `soft_delete_client()` RPC가 고객 + 연락처 + 확장을 함께 soft delete.

7. `client_msp_details` — 고객 MSP 확장 (1:1)

컬럼	타입	제약	설명
id	uuid	PK	
client_id	uuid	NOT NULL, UNIQUE, FK → clients (CASCADE)	

컬럼	타입	제약	설명
industry	industry_type	NULL	
company_size	company_size_type	NULL	
memo	text	NULL	
deleted_at	timestampz	NULL	

8. client_edu_details — 고객 교육 확장 (1:1)

컬럼	타입	제약	설명
id	uuid	PK	
client_id	uuid	NOT NULL, UNIQUE, FK → clients (CASCADE)	
edu_grade	text	NULL	
memo	text	NULL	
deleted_at	timestampz	NULL	

9. contacts — 고객 연락처

컬럼	타입	제약	설명
id	uuid	PK	
client_id	uuid	NOT NULL, FK → clients (CASCADE)	
name	text	NOT NULL	
email / phone / department / position / role	text	NULL	
is_primary	boolean	default false	대표 연락처 여부
deleted_at	timestampz	NULL	

인덱스: (id) WHERE deleted_at IS NULL · (client_id) · (name) gin_trgm

10. `contracts` — 계약 (통합 모델: MSP/교육/개발)

컬럼	타입	제약	설명
<code>id</code>	uuid	PK	
<code>contract_id</code>	text	NOT NULL, UNIQUE	표시 ID (<code>MSP-001</code> , <code>CT2026001</code>)
<code>client_id</code>	uuid	NOT NULL, FK → clients	
<code>type</code>	contract_type	NOT NULL	msp / edu / dev
<code>name</code>	text	NOT NULL	계약명 (헤더에서 인라인 수정, gin_trgm)
<code>memo</code>	text	NULL	
<code>total_amount</code>	bigint	default 0	계약 금액
<code>currency</code>	currency_type	default 'KRW'	
<code>stage</code>	text	CHECK (타입별)	파이프라인 단계 ↓
<code>assigned_to</code>	uuid	FK → profiles	사내 담당자
<code>contact_id</code>	uuid	FK → contacts	고객사 담당자
<code>created_at</code> / <code>updated_at</code>	timestampz	NOT NULL	
<code>deleted_at</code>	timestampz	NULL	soft delete

인덱스: `(id)` WHERE `deleted_at` IS NULL · `(client_id)` · `(name)` gin_trgm · `(stage)` · `(type)`

stage CHECK: `msp` = pre_contract → billing_complete → project_closed → unpaid · `edu` (교육) = proposal → contracted → operating → op_completed → settled · `dev` = 계약 없음 **삭제 가드:** 예치금 잔액 ≠ 0이면 `guard_contract_delete_with_deposit()` 트리거가 종료(soft delete)를 차단.

11. `contract_msp_details` — 계약 MSP 운영 상세 (1:1)

컬럼	타입	제약	설명
<code>id</code>	uuid	PK	
<code>contract_id</code>	uuid	NOT NULL, UNIQUE, FK → contracts (CASCADE)	
<code>credit_share</code>	credit_share_type	NULL	크레딧 세어

컬럼	타입	제약	설명
<code>expected_mrr</code>	bigint	NULL	예상 월 반복 매출
<code>payer</code>	payer_type	NULL	결제 주체
<code>aws_amount</code>	bigint	NULL	
<code>has_management_fee</code>	boolean	default false	
<code>billing_method</code>	billing_method_type	NULL	청구 방식
<code>sales_rep_id</code>	uuid	FK → employees	영업 담당
<code>aws_account_ids</code>	text[]	default '{}'	AWS 계정 ID 목록
<code>aws_account_search</code>	text	GENERATED STORED	<code>aws_account_ids</code> 배열→ 텍스트 (부분 검색용)
<code>aws_am</code>	text	NULL	AWS AM
<code>msp_grade</code>	msp_grade_type	NULL	None/FREE/MSP10/15/20/ETC
<code>billing_on</code> / <code>billing_on_alias</code>	boolean / text		빌링온 등록·별칭
<code>root_account_email</code>	text	NULL	루트 계정 메일
<code>tags</code>	text[]	default '{}'	
<code>deleted_at</code>	timestampz	NULL	

인덱스: `(aws_account_search) gin_trgm` — AWS 계정 ID 부분 검색

12. `contract_teams` — 팀별 매출 배분 (M:N)

컬럼	타입	제약	설명
<code>id</code>	uuid	PK	
<code>contract_id</code>	uuid	NOT NULL, FK → contracts (CASCADE)	
<code>team_id</code>	uuid	NOT NULL, FK → teams	
<code>percentage</code>	numeric(5,2)	CHECK (0 < p ≤ 100)	배분 비율
<code>deleted_at</code>	timestampz	NULL	

제약: `UNIQUE (contract_id, team_id)` 인덱스: `(contract_id)` · `(team_id)`

갱신은 `update_contract_teams(contract_id, allocations jsonb)` RPC로 전체 교체.

13. contract_tech_leads — 계약 담당 기술 (M:N, 복합 PK)

컬럼	타입	제약	설명
contract_id	uuid	PK, FK → contracts (CASCADE)	
employee_id	uuid	PK, FK → employees (CASCADE)	
created_at	timestampz	NOT NULL	

인덱스: (employee_id)

갱신은 `replace_contract_tech_leads(contract_id, employee_ids[])` RPC로 전체 교체.

14. contract_history — 계약 변경 이력 (단계 + 필드)

컬럼	타입	제약	설명
id	uuid	PK	
contract_id	uuid	NOT NULL, FK → contracts (CASCADE)	
from_stage / to_stage	text	NULL	단계 변경 시
field_name	text	default 'stage'	변경된 필드명 (예: '계약명', '금액')
old_value / new_value	text	NULL	필드 변경 전/후 값
changed_by	uuid	NOT NULL, FK → profiles	
note	text	NULL	
created_at	timestampz	NOT NULL	

인덱스: (contract_id)

단계 변경은 `change_contract_stage()` RPC, 일반 필드 변경은 앱의 `logChanges()` 가 기록.

15. deposit_accounts — 예치금 계좌 (계약당 활성 1개)

컬럼	타입	제약	설명
id	uuid	PK	
contract_id	uuid	NOT NULL, FK → contracts	

컬럼	타입	제약	설명
balance	bigint	NOT NULL, default 0	현재 잔액 (트리거 자동 계산)
total_deposit	bigint	NOT NULL, default 0	누적 입금 (트리거)
total_usage	bigint	NOT NULL, default 0	누적 사용 (트리거)
last_recalc_at	timestampz	NULL	마지막 재계산 시각
deleted_at	timestampz	NULL	비활성화

제약: `UNIQUE (contract_id) WHERE deleted_at IS NULL` (부분 unique — 계약당 활성 계좌 1개)

인덱스: `(contract_id) WHERE deleted_at IS NULL`

`balance = Σdeposit - Σusage + Σadjustment - Σrefund` (무효화 제외). 거래 변경 시 `recalc_deposit_account_balance()` 트리거가 재계산. 비활성화는 잔액 0 + `admin·c_level·team_lead`일 때만 (`assert_deposit_deactivation_safe()` 트리거).

16. deposit_transactions — 예치금 거래

컬럼	타입	제약	설명
id	uuid	PK	
account_id	uuid	NOT NULL, FK → deposit_accounts	
txn_date	date	NOT NULL	
txn_type	deposit_txn_type	NOT NULL	deposit / usage / adjustment / refund
amount	bigint	NOT NULL, CHECK	deposit·usage·refund > 0, adjustment ≠ 0
memo	text	NULL	
source	deposit_txn_source	NOT NULL, default 'manual'	manual / aws_api / billing_on
created_by	uuid	FK → profiles	
voided_at / voided_by / void_reason	timestampz / uuid / text	NULL	무효화 (삭제 대신 보존)
created_at	timestampz	NOT NULL	

인덱스: `(account_id, txn_date DESC) WHERE voided_at IS NULL` · `(account_id) WHERE voided_at IS NULL`

무효화는 행 삭제가 아니라 `voided_at` 기록. 비활성 계좌엔 거래 등록 불가(무효화만 허용, `assert_deposit_account_active()` 트리거). RLS: `deposit·usage INSERT`는 계약 접근자, `adjustment·refund INSERT` 및 `void`는 `admin·c_level·team_lead`.

17. `education_operations` — 교육 운영 (계약 하위)

컬럼	타입	제약	설명
<code>id</code>	<code>uuid</code>	PK	
<code>contract_id</code>	<code>uuid</code>	NOT NULL, FK → <code>contracts</code> (CASCADE)	
<code>operation_name</code>	<code>text</code>	NOT NULL	
<code>location</code> / <code>target_org</code>	<code>text</code>	NULL	
<code>start_date</code> / <code>end_date</code>	<code>date</code>	NULL	
<code>total_hours</code>	<code>numeric(6,1)</code>	NULL	
<code>contracted_count</code> / <code>recruited_count</code> / <code>actual_count</code>	<code>integer</code>	NULL	계약/모집/실제 인원
<code>provides_lunch</code> / <code>provides_snack</code>	<code>boolean</code>	default false	
<code>date_list</code>	<code>date[]</code>	default '{}'	교육 실시일 배열
<code>notes</code>	<code>text</code>	NULL	
<code>deleted_at</code>	<code>timestampz</code>	NULL	

인덱스: (`contract_id`)

18. `education_operation_dates` — 교육 일자별 상세

컬럼	타입	제약	설명
<code>id</code>	<code>uuid</code>	PK	
<code>operation_id</code>	<code>uuid</code>	NOT NULL, FK → <code>education_operations</code> (CASCADE)	
<code>education_date</code>	<code>date</code>	NOT NULL	
<code>hours</code>	<code>numeric</code>	default 0	

인덱스: (operation_id)

19. instructors — 강사 마스터 (내부·외부)

컬럼	타입	제약	설명
id	uuid	PK	
name	text	NOT NULL	
email / phone / organization / team / position	text	NULL	
status	text	default '활동'	

20. operation_instructors — 운영-강사 배정 (M:N)

컬럼	타입	제약	설명
id	uuid	PK	
operation_id	uuid	NOT NULL, FK → education_operations (CASCADE)	
instructor_id	uuid	NOT NULL, FK → instructors	
role	text	NOT NULL	주강사/보조 등
assigned_date	date	NULL	
notes	text	NULL	

인덱스: (instructor_id) · (operation_id)

ENUM 타입 (16개)

타입	값
user_role	staff, team_lead, admin, c_level
team_type	mss, tt, dev, ops, ai, ptn
business_type	mss, edu, dev
client_type	univ, corp, govt, asso, etc

타입	값
client_grade	A, B, C, D, E
client_status_type	신규, 진행중, 활성화, 휴면, 종료, 상태없음
industry_type	IT, 제조, 금융, 유통, 공공, 서울대 연구실, 기타
company_size_type	스타트업, 중소기업, 중견기업, 대기업, 공공기관
contract_type	msp, edu, dev
currency_type	KRW, USD
credit_share_type	가능, 불가능, 미정
msp_grade_type	None, FREE, MSP10, MSP15, MSP20, ETC
payer_type	ETV-AWS-13, ETV-AWS-14, Org-001, Billing Transfer
billing_method_type	대표님 직접 청구, 매월 10일 세금계산서 발행, 공공기관 별도 청구
deposit_txn_type	deposit, usage, adjustment, refund
deposit_txn_source	manual, aws_api, billing_on

뷰 (2개)

뷰	설명
client_list_view	clients (미삭제) + 활성화 계약 수 (contract_count). 고객 목록용
contracts_with_details	contracts (미삭제) + contract_msp_details + 고객명/표시ID + 사내담당자명 조인. 계약 목록·상세용

접근 제어 (RLS)

전 테이블 RLS 활성화. 핵심 판정 함수(SEcurity DEFINER):

함수	판정
user_role() / user_team_id()	현재 사용자의 role / 소속 팀
is_admin_or_clevel()	admin 또는 c_level
is_admin_clevel_or_lead()	admin · c_level · team_lead (예치금 관리)
can_access_client(id)	admin·c_level 전체 / 그 외 고객 business_types ∩ 내 팀 도메인 교집합 시

함수	판정
<code>can_access_contract(id)</code>	admin·c_level 전체 / 그 외 계약 type ∈ 내 팀 도메인 OR <code>contract_teams</code> 매핑(fallback)

- **clients** → SELECT 인증 사용자 전체 · INSERT는 admin·c_level 또는 내 팀 도메인이 신규 고객 업종 담당 시 · U/D `can_access_client`
- ****contacts/client_*.details**** → `can_access_client`
- **contracts** → SELECT·U/D `can_access_contract` · INSERT는 인증 사용자(이후 행 접근은 도메인 매핑으로 제한)
- **contract_*/education_operations/contract_history** → `can_access_contract`
- **operation_instructors** → 연결된 `education_operations` 의 계약 기준 `can_access_contract`
- **education_operation_dates** → SELECT 공개, C/U/D는 인증 사용자
- **deposit_accounts** → `can_access_contract` (SELECT·INSERT는 계약 type=msp 한정, UPDATE는 msp 무관)
- **deposit_transactions** → SELECT/INSERT는 계약의 `can_access_contract` ; deposit/usage 외 타입과 UPDATE는 admin·c_level·team_lead(또는 작성자 본인)
- **employees** → SELECT 공개, C·U·D는 admin만
- **profiles** → SELECT 공개, INSERT 인증 사용자, UPDATE 본인만 · **user_preferences** → 본인만
- **teams** → SELECT 공개
- **team_business_domains** → SELECT 인증 사용자
- **instructors** → SELECT 공개, INSERT/UPDATE 인증 사용자, DELETE 정책 없음

다단계 쓰기 RPC (트랜잭션 보장)

함수	역할	진입 가드
<code>create_contract_with_details(jsonb)</code>	계약 + MSP 상세 생성 + 고객 business_types 갱신 (ID 자동 발번)	<code>can_access_client</code>
<code>change_contract_stage(...)</code>	단계 변경 + 이력 기록	<code>can_access_contract</code>
<code>replace_contract_tech_leads(...)</code>	담당 기술 전체 교체	<code>can_access_contract</code>
<code>update_contract_teams(...)</code>	매출 배분 전체 교체	<code>can_access_contract</code>
<code>soft_delete_contract(id) /</code> <code>soft_delete_client(id)</code>	연관 행까지 일괄 soft delete	<code>can_access_contract</code> / <code>can_access_client</code>
<code>generate_client_id /</code> <code>generate_msp_contract_id /</code> <code>generate_edu_contract_id</code>	표시 ID 발번	없음 (단순 발번)

전부 SECURITY DEFINER → RLS 우회. 진입부 가드가 유일 방어선이며, `search_path` 는 `public`, `pg_temp` 로 고정한다.

인덱스 정책

PK·UNIQUE 컬럼은 자동 인덱스가 생성되므로 명시하지 않는다. 각 테이블의 **인덱스** 항목은 추가 인덱스만 표기한 것이며, 패턴은 다음과 같다:

- **부분 인덱스** (`WHERE deleted_at IS NULL` / `WHERE voided_at IS NULL`): 미삭제·유효 행만 인덱싱
— 소프트 삭제 필터 최적화
 - **gin_trgm**: `clients.name` , `contacts.name` , `contracts.name` ,
`contract_msp_details.aws_account_search` — 부분 문자열 검색
 - **부분 unique**: `deposit_accounts (contract_id) WHERE deleted_at IS NULL` — 계약당 활성
예치금 계좌 1개 보장
-

설계 특징

1. **통합 계약/고객 모델** → 타입(`mzp/edu/dev`)을 컬럼으로 두고 타입별 상세를 1:1 확장 테이블로 분리. 공통 조치는 단일 테이블, 타입 특화 필드는 확장.
2. **고객 삭제 없음** → soft delete만. 계약 종료는 stage로 관리.
3. **고객 계층 2단계 제한** → `check_client_hierarchy()` 트리거로 강제 (상위의 상위 금지).
4. **예치금 잔액은 파생값** → 직접 쓰지 않고 거래 합산 트리거로만 갱신. 무효화(void)는 행 삭제 대신 플래그.
5. **계약당 활성 예치금 계좌 1개** → 전역 UNIQUE 대신 부분 unique index (비활성 이력은 여러 개 허용).
6. **접근 제어** → 팀↔사업유형 도메인 매핑(`team_business_domains`) 기반. `contract_teams` 명시 매핑은 fallback.
7. **인덱스** → PK·UNIQUE 자동 인덱스는 명시 안 함. FK·검색(trgm)·정렬·소프트삭제 필터용만 추가.